SAP ABAP

OBJECT-ORIENTED PROGRAMMING

Developer's Guide to OOP in SAP ABAP

OBJECT-ORIENTED PROGRAMMING

INTRODUCTION

1. Object-Oriented Programming (OOP) is a programming paradigm based on the concept of objects.
2. OOP allows developers to create reusable, modular, and maintainable code.
3. ABAP Objects was introduced in SAP NetWeaver and is mandatory for modern SAP development.
4. Transaction code for class builder is SE24.
5. In OOP, we create classes which act as blueprints, and objects which are instances of classes.
6. OOP provides better structure for complex business logic across all SAP modules (FI, MM, SD, PP, HCM).
7. Modern SAP technologies like Fiori, RAP, CDS Views, and SAP BTP require OOP knowledge.

STEP-BY-STEP: CREATING A CLASS IN SAP

Step 1: Prerequisites

Before you start:

- You need authorization for transaction SE24 or SE80.
- Understanding of ABAP syntax basics.
- Knowledge of data types and structures.

Step 2: Create a Class (SE24 Transaction)

Purpose: A class is a template that defines attributes (data) and methods (behavior). It encapsulates data and functionality together.

Steps:

1. Go to T-code SE24 (Class Builder).

2. Enter a class name (e.g., ZCL_CUSTOMER_MANAGER) and click Create.
3. Choose Object Type:
    - **Class** (for normal classes)
    - **Interface** (for defining contracts)
    - **Exception Class** (for error handling)
4. Enter a short description (e.g., "Customer Management Class").
5. Click Save and assign to a package or local object ($TMP).

Step 3: Define Class Components

Purpose: Define attributes, methods, events, and other components of the class.

Steps:

1. In the **Properties** tab, set class properties:
    - **Instantiation:** Public, Protected, Private, or Abstract
    - **Final Indicator:** Check if class cannot be inherited
2. Go to the **Attributes** tab:
    - Define instance attributes (unique per object)
    - Define static attributes (shared across all objects)
    - Example: `MV_CUSTOMER_ID TYPE KUNNR`
3. Go to the **Methods** tab:
    - Define instance methods (operate on object data)
    - Define static methods (operate independently)
    - Define CONSTRUCTOR (automatically called when object is created)
    - Example: `GET_CUSTOMER_DETAILS` with IMPORTING/EXPORTING/RETURNING parameters
4. For each method, define:
    - Import parameters (input)
    - Export parameters (output)
    - Returning parameters (for functional methods)
    - Exceptions (error handling)

Step 4: Implement Methods

Steps:

1. Click on the method name in the Methods tab.
2. Click the **Code** button to open the method implementation editor.
3. Write your ABAP logic between:

```abap
METHOD method_name.
  "Your code here
ENDMETHOD.
```

4. Use `ME->` to refer to instance attributes within the method.
5. Example:

```abap
METHOD get_customer_details.
  SELECT SINGLE * FROM kna1
    INTO @DATA(ls_customer)
    WHERE kunnr = @iv_customer_id.

  IF sy-subrc = 0.
    rv_name = ls_customer-name1.
  ENDIF.
ENDMETHOD.
```

Step 5: Activate the Class

1. Click the **Activate** button (Ctrl+F3).
2. SAP checks syntax and activates the class.
3. Class is now ready to be used in ABAP programs.

Step 6: Use the Class in ABAP Program

Steps:

1. Create an object reference variable:

```abap
DATA: lo_customer TYPE REF TO zcl_customer_manager.
```

2. Create an instance (object):

```abap
CREATE OBJECT lo_customer.
```

3. Call methods:

```abap
DATA(lv_name) = lo_customer->get_customer_details(
  iv_customer_id = '0000100001' ).
WRITE: / 'Customer Name:', lv_name.
```

4. For static methods (no object creation needed):

```abap
zcl_utility=>get_system_date( ).
```

CORE OOP CONCEPTS

1. CLASSES AND OBJECTS

Class: Blueprint or template that defines structure and behavior **Object:** Instance of a class - actual entity in memory

Example:

```abap
```

```
CLASS lcl_employee DEFINITION.
  PUBLIC SECTION.
    DATA: mv_empid TYPE pernr_d,
          mv_name  TYPE string.
    METHODS: constructor IMPORTING iv_empid TYPE pernr_d,
             display_info.
ENDCLASS.

CLASS lcl_employee IMPLEMENTATION.
  METHOD constructor.
    mv_empid = iv_empid.
  ENDMETHOD.

  METHOD display_info.
    WRITE: / 'Employee ID:', mv_empid, 'Name:', mv_name.
  ENDMETHOD.
ENDCLASS.

"Usage:
DATA: lo_emp TYPE REF TO lcl_employee.
CREATE OBJECT lo_emp EXPORTING iv_empid = '00001234'.
lo_emp->display_info( ).
```

2. ENCAPSULATION

Definition: Bundling data and methods together while controlling access through visibility sections.

Visibility Levels:

- **PUBLIC:** Accessible from anywhere (outside the class)
- **PROTECTED:** Accessible within class and its subclasses only
- **PRIVATE:** Accessible only within the class itself

Benefits:

- Data hiding and protection
- Controlled access to sensitive information
- Better maintainability

Example:

```
abap
```

```
CLASS lcl_bank_account DEFINITION.
  PUBLIC SECTION.
    METHODS: deposit IMPORTING iv_amount TYPE p,
             get_balance RETURNING VALUE(rv_balance) TYPE p.

  PROTECTED SECTION.
    METHODS: validate_transaction.

  PRIVATE SECTION.
    DATA: mv_balance TYPE p DECIMALS 2,
          mv_account_number TYPE string.
    METHODS: update_ledger.
ENDCLASS.
```

3. INHERITANCE

Definition: Creating new classes based on existing classes, inheriting attributes and methods.

Types:

- Single Inheritance (one parent class)
- Multi-level Inheritance (chain of inheritance)

Keyword: INHERITING FROM

Benefits:

- Code reusability
- Hierarchical classification
- Method overriding for specialized behavior

Example:

```abap
```

```
"Parent Class
CLASS lcl_document DEFINITION.
  PUBLIC SECTION.
    DATA: mv_docnum TYPE belnr_d,
          mv_docdate TYPE datum.
    METHODS: display_header.
ENDCLASS.

"Child Class - Invoice
CLASS lcl_invoice DEFINITION INHERITING FROM lcl_document.
  PUBLIC SECTION.
    DATA: mv_customer TYPE kunnr,
          mv_amount TYPE wrbtr.
    METHODS: display_header REDEFINITION,  "Override parent method
             calculate_tax RETURNING VALUE(rv_tax) TYPE wrbtr.
ENDCLASS.

"Child Class - Delivery
CLASS lcl_delivery DEFINITION INHERITING FROM lcl_document.
  PUBLIC SECTION.
    DATA: mv_ship_to TYPE kunnr,
          mv_weight TYPE ntgew.
    METHODS: display_header REDEFINITION,
             calculate_freight RETURNING VALUE(rv_freight) TYPE wrbtr.
ENDCLASS.
```

Real-world Application:

- FI Module: Base class for accounting documents, subclasses for invoices, credit memos, payments
- MM Module: Base material class, specialized classes for raw materials, finished goods, services
- SD Module: Base order class, specific classes for standard orders, rush orders, returns

4. POLYMORPHISM

Definition: The ability to process objects differently based on their data type or class.

Types:

- **Static Polymorphism:** Method overloading (not directly supported in ABAP)
- **Dynamic Polymorphism:** Method overriding through inheritance

Benefits:

- Single interface for different implementations

- Flexibility in code design
- Easy to extend functionality

Example:

abap

```
"Base Class
CLASS lcl_payment DEFINITION.
  PUBLIC SECTION.
    METHODS: process_payment ABSTRACT
             IMPORTING iv_amount TYPE p
             RETURNING VALUE(rv_success) TYPE abap_bool.
ENDCLASS.

"Credit Card Payment
CLASS lcl_credit_card DEFINITION INHERITING FROM lcl_payment.
  PUBLIC SECTION.
    METHODS: process_payment REDEFINITION.
ENDCLASS.

CLASS lcl_credit_card IMPLEMENTATION.
  METHOD process_payment.
    "Credit card specific logic
    WRITE: / 'Processing credit card payment:', iv_amount.
    rv_success = abap_true.
  ENDMETHOD.
ENDCLASS.

"Wire Transfer Payment
CLASS lcl_wire_transfer DEFINITION INHERITING FROM lcl_payment.
  PUBLIC SECTION.
    METHODS: process_payment REDEFINITION.
ENDCLASS.

CLASS lcl_wire_transfer IMPLEMENTATION.
  METHOD process_payment.
    "Wire transfer specific logic
    WRITE: / 'Processing wire transfer:', iv_amount.
    rv_success = abap_true.
  ENDMETHOD.
ENDCLASS.

"Polymorphic usage:
DATA: lo_payment TYPE REF TO lcl_payment,
      lv_type TYPE string VALUE 'CREDIT_CARD'.

IF lv_type = 'CREDIT_CARD'.
  CREATE OBJECT lo_payment TYPE lcl_credit_card.
ELSEIF lv_type = 'WIRE'.
  CREATE OBJECT lo_payment TYPE lcl_wire_transfer.
ENDIF.
```

```
lo_payment->process_payment( iv_amount = 1000 ).
```

5. ABSTRACTION

Definition: Hiding complex implementation details and showing only essential features.

Implementation Methods:

- **Abstract Classes:** Cannot be instantiated, must be inherited
- **Interfaces:** Pure contracts defining method signatures

Abstract Class Example:

```
abap

CLASS lcl_shape DEFINITION ABSTRACT.
  PUBLIC SECTION.
    METHODS: calculate_area ABSTRACT
               RETURNING VALUE(rv_area) TYPE p,
             display_info.
ENDCLASS.

CLASS lcl_circle DEFINITION INHERITING FROM lcl_shape.
  PUBLIC SECTION.
    DATA: mv_radius TYPE p.
    METHODS: calculate_area REDEFINITION.
ENDCLASS.

CLASS lcl_rectangle DEFINITION INHERITING FROM lcl_shape.
  PUBLIC SECTION.
    DATA: mv_length TYPE p,
          mv_width TYPE p.
    METHODS: calculate_area REDEFINITION.
ENDCLASS.
```

Interface Example:

```
abap
```

```
INTERFACE lif_printable.
  METHODS: print,
           get_printer_name RETURNING VALUE(rv_name) TYPE string.
ENDINTERFACE.

INTERFACE lif_emailable.
  METHODS: send_email IMPORTING iv_recipient TYPE string,
           validate_email RETURNING VALUE(rv_valid) TYPE abap_bool.
ENDINTERFACE.

CLASS lcl_invoice DEFINITION.
  PUBLIC SECTION.
    INTERFACES: lif_printable,
                lif_emailable.
ENDCLASS.

CLASS lcl_invoice IMPLEMENTATION.
  METHOD lif_printable~print.
    WRITE: / 'Printing invoice...'.
  ENDMETHOD.

  METHOD lif_printable~get_printer_name.
    rv_name = 'HP_LASER_001'.
  ENDMETHOD.

  METHOD lif_emailable~send_email.
    WRITE: / 'Sending invoice to:', iv_recipient.
  ENDMETHOD.

  METHOD lif_emailable~validate_email.
    IF iv_recipient CS '@'.
      rv_valid = abap_true.
    ENDIF.
  ENDMETHOD.
ENDCLASS.
```

TYPES OF CLASSES

1. Normal Class

Standard class that can be instantiated and used directly.

2. Abstract Class

- Cannot be instantiated directly
- Must be inherited by subclasses
- Contains at least one abstract method
- Keyword: `ABSTRACT` in class definition
- Use case: Base class for common functionality

3. Final Class

- Cannot be inherited
- Keyword: `FINAL` in class definition
- Use case: Prevent modification of critical logic

4. Singleton Class

- Only one instance can exist
- Instance is globally accessible
- Implementation uses static attribute and method
- Use case: Configuration managers, logging services

Example:

```abap
```

```
CLASS lcl_config_manager DEFINITION FINAL.
  PUBLIC SECTION.
    CLASS-METHODS: get_instance
                   RETURNING VALUE(ro_instance) TYPE REF TO lcl_config_manager.
    METHODS: get_config IMPORTING iv_key TYPE string
                     RETURNING VALUE(rv_value) TYPE string.

  PRIVATE SECTION.
    CLASS-DATA: go_instance TYPE REF TO lcl_config_manager.
    METHODS: constructor.
ENDCLASS.

CLASS lcl_config_manager IMPLEMENTATION.
  METHOD get_instance.
    IF go_instance IS NOT BOUND.
      CREATE OBJECT go_instance.
    ENDIF.
    ro_instance = go_instance.
  ENDMETHOD.

  METHOD constructor.
    "Initialize configuration
  ENDMETHOD.
ENDCLASS.
```

INTERFACES IN OOP

What is an Interface?

An interface is a contract that defines method signatures without implementation. Classes implementing the interface must provide the actual implementation.

Characteristics:

1. Interfaces contain only method definitions (no implementation)
2. A class can implement multiple interfaces
3. Promotes loose coupling
4. Enforces consistent behavior across different classes

Syntax:

abap

```abap
INTERFACE lif_interface_name.
  METHODS: method1,
        method2 IMPORTING iv_param TYPE type.
ENDINTERFACE.
```

Implementation:

```abap
CLASS lcl_class_name DEFINITION.
  PUBLIC SECTION.
    INTERFACES: lif_interface_name.
ENDCLASS.

CLASS lcl_class_name IMPLEMENTATION.
  METHOD lif_interface_name~method1.
    "Implementation code
  ENDMETHOD.

  METHOD lif_interface_name~method2.
    "Implementation code
  ENDMETHOD.
ENDCLASS.
```

Real-World Example - Payment Processing:

```abap
```

```
INTERFACE lif_payment_processor.
  METHODS: authorize_payment IMPORTING iv_amount TYPE p
                      RETURNING VALUE(rv_auth_code) TYPE string,
           capture_payment IMPORTING iv_auth_code TYPE string
                      RETURNING VALUE(rv_success) TYPE abap_bool,
           refund_payment IMPORTING iv_transaction_id TYPE string
                      RETURNING VALUE(rv_refund_id) TYPE string.
ENDINTERFACE.

CLASS lcl_stripe_payment DEFINITION.
  PUBLIC SECTION.
    INTERFACES: lif_payment_processor.
ENDCLASS.

CLASS lcl_paypal_payment DEFINITION.
  PUBLIC SECTION.
    INTERFACES: lif_payment_processor.
ENDCLASS.
```

INSTANCE VS STATIC COMPONENTS

Feature	Instance Components	Static Components
Keyword	DATA, METHODS	CLASS-DATA, CLASS-METHODS
Access	Via object reference	Via class name directly
Memory	Separate for each object	Shared across all objects
Syntax	`lo_obj->method( )`	`zcl_class=>method( )`
Use Case	Object-specific data/behavior	Utility functions, counters

Example:

```abap
```

```
CLASS lcl_counter DEFINITION.
  PUBLIC SECTION.
    CLASS-DATA: gv_total_count TYPE i.
    DATA: mv_instance_count TYPE i.

    CLASS-METHODS: get_total_count RETURNING VALUE(rv_count) TYPE i.
    METHODS: constructor,
             get_instance_count RETURNING VALUE(rv_count) TYPE i.
ENDCLASS.

CLASS lcl_counter IMPLEMENTATION.
  METHOD constructor.
    gv_total_count = gv_total_count + 1.
    mv_instance_count = gv_total_count.
  ENDMETHOD.

  METHOD get_total_count.
    rv_count = gv_total_count.
  ENDMETHOD.

  METHOD get_instance_count.
    rv_count = mv_instance_count.
  ENDMETHOD.
ENDCLASS.

"Usage:
DATA: lo_obj1 TYPE REF TO lcl_counter,
      lo_obj2 TYPE REF TO lcl_counter.

CREATE OBJECT lo_obj1.
CREATE OBJECT lo_obj2.

WRITE: / 'Total objects created:', lcl_counter=>get_total_count( ).
WRITE: / 'Object 1 instance:', lo_obj1->get_instance_count( ).
WRITE: / 'Object 2 instance:', lo_obj2->get_instance_count( ).
```

CONSTRUCTORS IN OOP

Types of Constructors:

1. **Instance Constructor (CONSTRUCTOR)**
    - Automatically called when object is created
    - Used to initialize instance attributes

- Can have importing parameters
- Can raise exceptions

2. **Static Constructor (CLASS_CONSTRUCTOR)**
    - Called automatically once when class is first accessed
    - No parameters allowed
    - Cannot raise exceptions
    - Used to initialize static attributes

Example:

abap

```
CLASS lcl_employee DEFINITION.
  PUBLIC SECTION.
    CLASS-DATA: gv_company_name TYPE string.
    DATA: mv_empid TYPE pernr_d,
          mv_name TYPE string,
          mv_salary TYPE p.

    CLASS-METHODS: class_constructor.
    METHODS: constructor IMPORTING iv_empid TYPE pernr_d
                                   iv_name TYPE string
                         RAISING cx_parameter_error.
ENDCLASS.

CLASS lcl_employee IMPLEMENTATION.
  METHOD class_constructor.
    gv_company_name = 'ABC Corporation'.
  ENDMETHOD.

  METHOD constructor.
    IF iv_empid IS INITIAL.
      RAISE EXCEPTION TYPE cx_parameter_error.
    ENDIF.

    mv_empid = iv_empid.
    mv_name = iv_name.

    "Fetch salary from database
    SELECT SINGLE salary FROM pa0008
      INTO @mv_salary
      WHERE pernr = @iv_empid.
  ENDMETHOD.
ENDCLASS.
```

EXCEPTION HANDLING WITH CLASSES

Exception Class Hierarchy:

1. **CX_ROOT** - Base class for all exceptions
    - **CX_STATIC_CHECK** - Must be declared in method signature
    - **CX_DYNAMIC_CHECK** - Can be caught at runtime
    - **CX_NO_CHECK** - Runtime errors, cannot be handled

Creating Custom Exception Class:

Steps:

1. Go to SE24
2. Create exception class (e.g., ZCX_CUSTOM_ERROR)
3. Choose parent class (usually CX_STATIC_CHECK)
4. Implement interface IF_T100_MESSAGE for message texts
5. Add custom attributes if needed

Example:

```abap
```

```
"Define exception class
CLASS zcx_order_error DEFINITION PUBLIC
  INHERITING FROM cx_static_check.

  PUBLIC SECTION.
    INTERFACES: if_t100_message.
    DATA: mv_order_id TYPE vbeln,
          mv_error_text TYPE string.

    METHODS: constructor IMPORTING iv_order_id TYPE vbeln
                                   iv_text TYPE string.
ENDCLASS.

"Business logic class
CLASS lcl_order_processor DEFINITION.
  PUBLIC SECTION.
    METHODS: create_order IMPORTING is_order TYPE sales_order
                         RAISING zcx_order_error.
ENDCLASS.

CLASS lcl_order_processor IMPLEMENTATION.
  METHOD create_order.
    IF is_order-customer IS INITIAL.
      RAISE EXCEPTION TYPE zcx_order_error
        EXPORTING iv_order_id = is_order-order_id
                  iv_text = 'Customer is mandatory'.
    ENDIF.

    "Create order logic
  ENDMETHOD.
ENDCLASS.

"Usage:
DATA: lo_processor TYPE REF TO lcl_order_processor.
CREATE OBJECT lo_processor.

TRY.
    lo_processor->create_order( ls_order ).
    WRITE: / 'Order created successfully'.
  CATCH zcx_order_error INTO DATA(lx_error).
    WRITE: / 'Error:', lx_error->mv_error_text.
ENDTRY.
```

OOP IN SAP MODULES

1. Financial Accounting (FI)

Application: Document Posting Framework

```abap
CLASS lcl_fi_document DEFINITION ABSTRACT.
  PUBLIC SECTION.
    DATA: ms_header TYPE bkpf.
    METHODS: validate ABSTRACT RETURNING VALUE(rv_valid) TYPE abap_bool,
             post RETURNING VALUE(rv_docnum) TYPE belnr_d,
             reverse IMPORTING iv_reason TYPE string.
ENDCLASS.

CLASS lcl_invoice DEFINITION INHERITING FROM lcl_fi_document.
  PUBLIC SECTION.
    METHODS: validate REDEFINITION,
             calculate_tax RETURNING VALUE(rv_tax) TYPE wrbtr.
  PRIVATE SECTION.
    DATA: mt_line_items TYPE TABLE OF bseg.
ENDCLASS.
```

2. Materials Management (MM)

Application: Material Master Processing

```abap
CLASS lcl_material DEFINITION ABSTRACT.
  PUBLIC SECTION.
    DATA: mv_matnr TYPE matnr,
          mv_mtart TYPE mtart.
    METHODS: create_material ABSTRACT,
             extend_to_plant ABSTRACT IMPORTING iv_werks TYPE werks_d.
ENDCLASS.

CLASS lcl_raw_material DEFINITION INHERITING FROM lcl_material.
  PUBLIC SECTION.
    METHODS: create_material REDEFINITION,
             extend_to_plant REDEFINITION.
    DATA: mv_reorder_point TYPE eisbe.
ENDCLASS.
```

3. Sales & Distribution (SD)

Application: Order Processing Pipeline

```abap
INTERFACE lif_order_processor.
  METHODS: validate_order,
           check_credit_limit,
           check_availability,
           create_delivery,
           create_invoice.
ENDINTERFACE.

CLASS lcl_standard_order DEFINITION.
  PUBLIC SECTION.
    INTERFACES: lif_order_processor.
ENDCLASS.

CLASS lcl_rush_order DEFINITION.
  PUBLIC SECTION.
    INTERFACES: lif_order_processor.
    METHODS: expedite_shipping.
ENDCLASS.
```

4. Production Planning (PP)

Application: Manufacturing Execution

```abap
CLASS lcl_production_order DEFINITION.
  PUBLIC SECTION.
    METHODS: create_order,
             release_order,
             confirm_operations,
             goods_receipt.
    DATA: mv_aufnr TYPE aufnr,
          mo_bom TYPE REF TO lcl_bom_manager,
          mo_routing TYPE REF TO lcl_routing_manager.
ENDCLASS.
```

5. Human Capital Management (HCM)

Application: Employee Hierarchy

```abap
```

```
CLASS lcl_employee DEFINITION.
  PUBLIC SECTION.
    DATA: mv_pernr TYPE pernr_d,
          mv_name TYPE emnam,
          mo_manager TYPE REF TO lcl_employee,
          mt_subordinates TYPE TABLE OF REF TO lcl_employee.

    METHODS: get_reporting_chain RETURNING VALUE(rt_chain) TYPE tt_pernr,
             calculate_team_size RETURNING VALUE(rv_size) TYPE i.
ENDCLASS.
```

OOP WITH SAP HANA

AMDP (ABAP Managed Database Procedures)

AMDP allows writing database procedures in ABAP classes that execute directly on HANA.

Syntax:

```
abap
```

```
CLASS lcl_hana_calculations DEFINITION.
  PUBLIC SECTION.
    INTERFACES: if_amdp_marker_hdb.

    CLASS-METHODS: get_sales_summary
      IMPORTING VALUE(iv_year) TYPE gjahr
      EXPORTING VALUE(et_results) TYPE tt_sales.
ENDCLASS.

CLASS lcl_hana_calculations IMPLEMENTATION.
  METHOD get_sales_summary BY DATABASE PROCEDURE
                      FOR HDB
                      LANGUAGE SQLSCRIPT
                      OPTIONS READ-ONLY
                      USING vbrk vbrp.

    et_results = SELECT
                   vbrk.vkorg,
                   vbrk.vtweg,
                   SUM(vbrp.netwr) AS total_revenue,
                   COUNT(DISTINCT vbrk.vbeln) AS order_count
                 FROM vbrk
                 INNER JOIN vbrp ON vbrk.vbeln = vbrp.vbeln
                 WHERE vbrk.gjahr = :iv_year
                 GROUP BY vbrk.vkorg, vbrk.vtweg;
  ENDMETHOD.
ENDCLASS.
```

Benefits:

- Executes at database layer (faster)
- Leverages HANA's columnar storage
- Reduces data transfer between layers

DESIGN PATTERNS IN OOP

1. Factory Pattern

Creates objects without specifying exact class.

```
abap
```

```
CLASS lcl_document_factory DEFINITION.
  PUBLIC SECTION.
    CLASS-METHODS: create_document
                   IMPORTING iv_type TYPE string
                   RETURNING VALUE(ro_doc) TYPE REF TO lcl_document.
ENDCLASS.

CLASS lcl_document_factory IMPLEMENTATION.
  METHOD create_document.
    CASE iv_type.
      WHEN 'INVOICE'.
        CREATE OBJECT ro_doc TYPE lcl_invoice.
      WHEN 'DELIVERY'.
        CREATE OBJECT ro_doc TYPE lcl_delivery.
      WHEN 'ORDER'.
        CREATE OBJECT ro_doc TYPE lcl_order.
    ENDCASE.
  ENDMETHOD.
ENDCLASS.
```

2. Singleton Pattern

Ensures only one instance exists.

abap

```
CLASS lcl_logger DEFINITION FINAL.
  PUBLIC SECTION.
    CLASS-METHODS: get_instance RETURNING VALUE(ro_instance) TYPE REF TO lcl_logger.
    METHODS: log_message IMPORTING iv_message TYPE string.

  PRIVATE SECTION.
    CLASS-DATA: go_instance TYPE REF TO lcl_logger.
    METHODS: constructor.
ENDCLASS.
```

3. Observer Pattern

One object notifies multiple dependent objects of state changes.

abap

```
CLASS lcl_subject DEFINITION.
  PUBLIC SECTION.
    METHODS: attach IMPORTING io_observer TYPE REF TO lif_observer,
             detach IMPORTING io_observer TYPE REF TO lif_observer,
             notify_all.
  PRIVATE SECTION.
    DATA: mt_observers TYPE TABLE OF REF TO lif_observer.
ENDCLASS.
```

4. Strategy Pattern

Defines family of algorithms, encapsulates each one, makes them interchangeable.

abap

```
INTERFACE lif_pricing_strategy.
  METHODS: calculate_price IMPORTING iv_base_price TYPE p
                   RETURNING VALUE(rv_final_price) TYPE p.
ENDINTERFACE.

CLASS lcl_retail_pricing DEFINITION.
  PUBLIC SECTION.
    INTERFACES: lif_pricing_strategy.
ENDCLASS.

CLASS lcl_wholesale_pricing DEFINITION.
  PUBLIC SECTION.
    INTERFACES: lif_pricing_strategy.
ENDCLASS.
```

OOP VS PROCEDURAL - QUICK COMPARISON

Feature	Procedural ABAP	Object-Oriented ABAP
Approach	Function-based	Object-based
Code Organization	Programs, Function Modules	Classes, Methods
Reusability	Limited	High
Maintainability	Difficult for large projects	Easier with encapsulation
Data & Logic	Separate	Bundled together
Inheritance	Not supported	Fully supported

Feature	Procedural ABAP	Object-Oriented ABAP
Polymorphism	Not possible	Supported
Modern SAP	Legacy support only	Mandatory (Fiori, RAP, BTP)
Transaction	SE38, SE37	SE24, SE80
S/4HANA	Being phased out	Required

OOP IN MODERN SAP TECHNOLOGIES

1. SAP Fiori / UI5

Backend services for Fiori apps are built using OOP.

```abap
CLASS zcl_sales_service DEFINITION PUBLIC
  INHERITING FROM /iwbep/cl_mgw_push_abs_data.

  PUBLIC SECTION.
    METHODS: /iwbep/if_mgw_appl_srv_runtime~get_entityset REDEFINITION,
             /iwbep/if_mgw_appl_srv_runtime~create_entity REDEFINITION.
ENDCLASS.
```

2. RAP (RESTful ABAP Programming)

Behavior implementation uses OOP principles.

```abap
CLASS lhc_sales DEFINITION INHERITING FROM cl_abap_behavior_handler.
  PRIVATE SECTION.
    METHODS: create FOR MODIFY,
             update FOR MODIFY,
             delete FOR MODIFY,
             read FOR READ.
ENDCLASS.
```

3. CDS Views with OOP

Consume CDS views in OOP classes for business logic.

```abap
```

```
CLASS lcl_analytics DEFINITION.
  PUBLIC SECTION.
    METHODS: get_sales_data IMPORTING iv_customer TYPE kunnr
                    RETURNING VALUE(rt_data) TYPE tt_sales.
ENDCLASS.

CLASS lcl_analytics IMPLEMENTATION.
  METHOD get_sales_data.
    SELECT * FROM z_cds_sales_view
      WHERE customer = @iv_customer
      INTO TABLE @rt_data.
  ENDMETHOD.
ENDCLASS.
```

4. SAP BTP Integration

Integration with Cloud Foundry, APIs using HTTP client classes.

```
abap

CLASS lcl_api_connector DEFINITION.
  PUBLIC SECTION.
    METHODS: call_external_api IMPORTING iv_endpoint TYPE string
                        RETURNING VALUE(rv_response) TYPE string.
  PRIVATE SECTION.
    DATA: mo_http_client TYPE REF TO if_http_client.
ENDCLASS.
```

BEST PRACTICES FOR OOP IN SAP

1. Naming Conventions

- Global classes: `ZCL_` or `YCL_` prefix
- Local classes: `LCL_` prefix
- Interfaces: `ZIF_` or `LIF_` prefix
- Exception classes: `ZCX_` prefix

2. Design Principles (SOLID)

- **S**ingle Responsibility: One class, one purpose
- **O**pen/Closed: Open for extension, closed for modification

- **L**iskov Substitution: Subclasses should be substitutable
- **I**nterface Segregation: Many specific interfaces better than one general
- **D**ependency Inversion: Depend on abstractions, not concrete classes

3. Performance Tips

- Use static methods for utility functions
- Avoid deep inheritance chains (max 3-4 levels)
- Implement lazy loading for heavy objects
- Use object pooling for frequently created objects

4. Documentation

- Add class/method descriptions
- Use inline comments for complex logic
- Document exceptions that can be raised

5. Testing

- Create unit test classes (ABAP Unit)
- Use test seams for dependency injection
- Maintain test coverage above 80%

KEY TRANSACTION CODES

T-Code	Description
SE24	Class Builder (Main tool for OOP)
SE80	Object Navigator (Integrated development)
SE19	BADI Implementation
SE37	Function Builder (Procedural)
SICF	HTTP Service Configuration (Web services)

CONCLUSION

Why OOP is Essential in SAP:

1. **Mandatory for S/4HANA:** New features require OOP knowledge
2. **Better Code Quality:** Encapsulation, reusability, maintainability
3. **Modern Frameworks:** Fiori, RAP, BTP all built on OOP
4. **Career Growth:** OOP skills are highly valued in SAP market
5. **Cross-Module:** Single OOP concept applies to all modules (FI, MM, SD, PP, HCM)
6. **Future-Proof:** SAP moving away from procedural programming

Next Steps:

1. Practice creating classes in SE24
2. Study SAP standard classes (CL_* classes)
3. Implement design patterns in real projects
4. Learn AMDP for HANA optimization
5. Explore RAP for modern application development
6. Build Fiori services using OOP backend

Document Version: 1.0
Last Updated: December 2024
Author: SAP ABAP Development Team

This guide covers OOP concepts from basics to advanced topics. For latest SAP documentation, refer to help.sap.com